CSA1702-ARTIFICIAL INTELLIGENCE

LAB EXPERIMENTS

Program 1 : Python Program to Solve 8-Puzzle Problem

Aim:

To develop a Python program to solve the 8-Puzzle Problem using a

Algorithm – 8-Puzzle Problem:

1. search algorithm **Breadth-First Search (BFS)**

2. Represent the puzzle as a **3 × 3 matrix**.

3. Define the **goal state** as (1,2,3,4,5,6,7,8,0).

4. Locate the empty tile 0.

5. Generate all possible moves: **Up, Down, Left, and Right**.

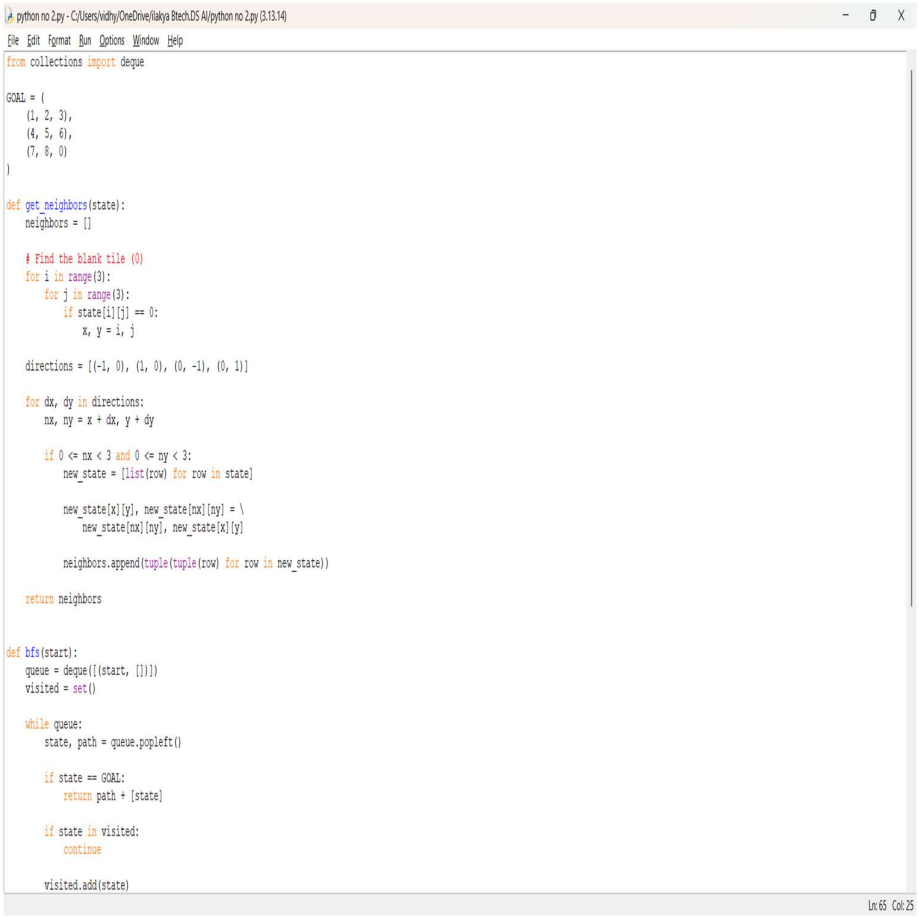
6. Use **Breadth-First Search (BFS)** to explore the possible states.

7. Store the visited states to avoid repeated states.

8. If the goal state is reached, display the solution path.

9. If all possible states are explored without reaching the goal, display **"No solution exists."**

Coding:



```
python no 2.py - C:/Users/vidhy/OneDrive/ilakya Btech.DS AI/python no 2.py (3.13.14)
File  Edit  Format  Run  Options  Window  Help
from collections import deque

GOAL = (
    (1, 2, 3),
    (4, 5, 6),
    (7, 8, 0)
)

def get_neighbors(state):
    neighbors = []

    # Find the blank tile (0)
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                x, y = i, j

    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dx, dy in directions:
        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [list(row) for row in state]

            new_state[x][y], new_state[nx][ny] = \
                new_state[nx][ny], new_state[x][y]

            neighbors.append(tuple(tuple(row) for row in new_state))

    return neighbors


def bfs(start):
    queue = deque([(start, [])])
    visited = set()

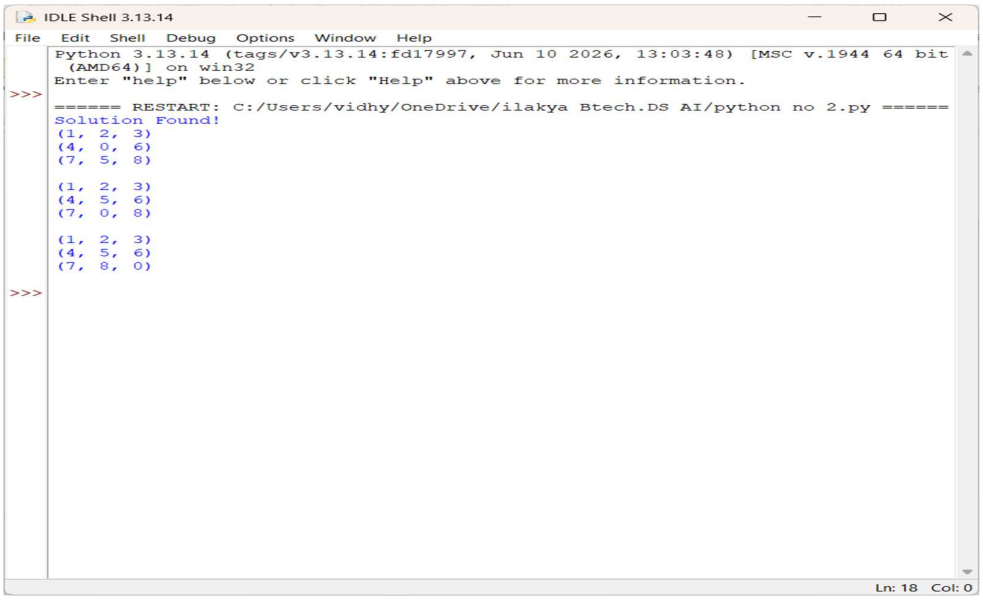
    while queue:
        state, path = queue.popleft()

        if state == GOAL:
            return path + [state]

        if state in visited:
            continue

        visited.add(state)
Ln: 65  Col: 25
```

Output:



```
IDLE Shell 3.13.14
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit
 (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
====== RESTART: C:/Users/vidhy/OneDrive/ilakya Btech.DS AI/python no 2.py ======
Solution Found!
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

>>>
Ln: 18  Col: 0
```

Result: The 8- Puzzle Program solved successfully using Python

Program 2: Python Program to Solve 8-Queen Problem

Aim:

To develop a Python program to solve the 8-Queen Problem using the Backtracking technique.

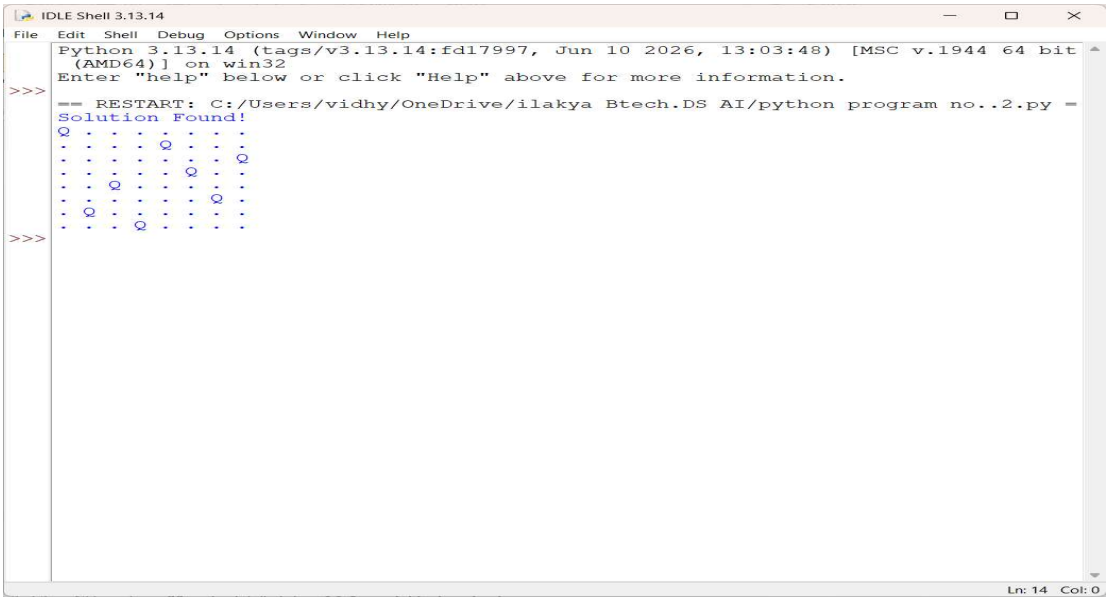
Algorithm – 8-Queen Problem:

1. Create an **8 × 8 chessboard**.
2. Place a queen in the first row.
3. Check whether the queen can be placed safely in the selected column.
4. Check that no other queen is present in the **same column**.
5. Check that no other queen is present on the **same diagonal**.
6. If the position is safe, place the queen and move to the next row.
7. If no safe position is available, **backtrack** to the previous row.
8. Continue placing queens until all **8 queens** are placed.
9. Display the solution.

Coding:



Output:



Result:

The 8-Queen Problem was solved successfully using Python.

Program 3: Python program to solve Water Jug Problem.

Aim:

To write a Python program to solve the Water Jug Problem using state transitions.

Algorithm:

1. Initialize both jugs as empty, \(x = 0\) and \(y = 0\).
2. Set the capacities of the two jugs as 4 litres and 3 litres.
3. Check whether the target quantity of 2 litres is reached.
4. If the first jug is empty, fill it completely.
5. If the second jug is full, empty it.
6. Otherwise, pour water from the first jug into the second jug until the second jug is full or the first jug is empty.
7. Display the current state of both jugs after each operation.
8. Repeat steps 4–8 until the target quantity is reached.
9. Display "Target achieved!"

Coding:

program no 3.py - C:/Users/vidhy/program no 3.py (3.13.14)

File Edit Format Run Options Window Help

```
def water_jug(capacity1, capacity2, target):
    x = 0
    y = 0

    print("Initial State:", (x, y))

    while x != target and y != target:
        if x == 0:
            x = capacity1
        elif y == capacity2:
            y = 0
        else:
            transfer = min(x, capacity2 - y)
            x = x - transfer
            y = y + transfer

        print("Current State:", (x, y))

    print("Target achieved!")

water_jug(4, 3, 2)
```

Output:

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==================== RESTART: C:/Users/vidhy/program no 3.py ====================
Initial State: (0, 0)
Current State: (4, 0)
Current State: (1, 3)
Current State: (1, 0)
Current State: (0, 1)
Current State: (4, 1)
Current State: (2, 3)
Target achieved!
>>>
```

Result:

Thus, the Water Jug Problem was successfully implemented in Python, and the target quantity of 2 litres was obtained.

Program 4: Crypt-Arithmetic Problem

Aim: To solve a crypt-arithmetic puzzle in which each letter represents a unique digit.

Algorithm:

1. Start the program.
2. Assign different digits to each letter.
3. Check whether the arithmetic equation is correct.
4. Display the valid digit assignment.
5. Stop the program.

Coding:

program no 4.py - C:/Users/vidhy/program no 4.py (3.13.14)

File Edit Format Run Options Window Help

```
from itertools import permutations

word1 = "SEND"
word2 = "MORE"
result = "MONEY"

letters = list(dict.fromkeys(word1 + word2 + result))

for digits in permutations(range(10), len(letters)):
    d = dict(zip(letters, digits))

    if d[word1[0]] == 0 or d[word2[0]] == 0:
        continue
    if d[result[0]] == 0:
        continue

    a = int("".join(str(d[c]) for c in word1))
    b = int("".join(str(d[c]) for c in word2))
    c = int("".join(str(d[x]) for x in result))

    if a + b == c:
        print(a, "+", b, "=", c)
        print(d)
        break
```

Output:

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/vidhy/program no 4.py ==========
9567 + 1085 = 10652
{'S': 9, 'E': 5, 'N': 6, 'D': 7, 'M': 1, 'O': 0, 'R': 8, 'Y': 2}
>>>
```

Result: Thus, the crypt-arithmetic problem was solved successfully using Python.

Program 5: Missionaries and Cannibals Problem

Aim:

To transport three missionaries and three cannibals across a river without allowing cannibals to outnumber missionaries on either bank when missionaries are present.

Algorithm:

1. Start with three missionaries and three cannibals on the left bank.
2. Use a boat that can carry one or two people.
3. Generate possible boat crossings.
4. Check that no missionaries are outnumbered by cannibals on either bank.
5. Repeat until everyone reaches the right bank.
6. Display the solution and stop.

Coding:

program 5.py - C:/Users/vidhy/program 5.py (3.13.14)

File Edit Format Run Options Window Help

```
from collections import deque

start = (3, 3, 0)
goal = (0, 0, 1)

moves = [(1, 0), (2, 0), (0, 1),
         (0, 2), (1, 1)]

queue = deque([(start, [start])])
visited = {start}

def valid(m, c):
    mr, cr = 3 - m, 3 - c

    return (0 <= m <= 3 and 0 <= c <= 3
            and (m == 0 or m >= c)
            and (mr == 0 or mr >= cr))

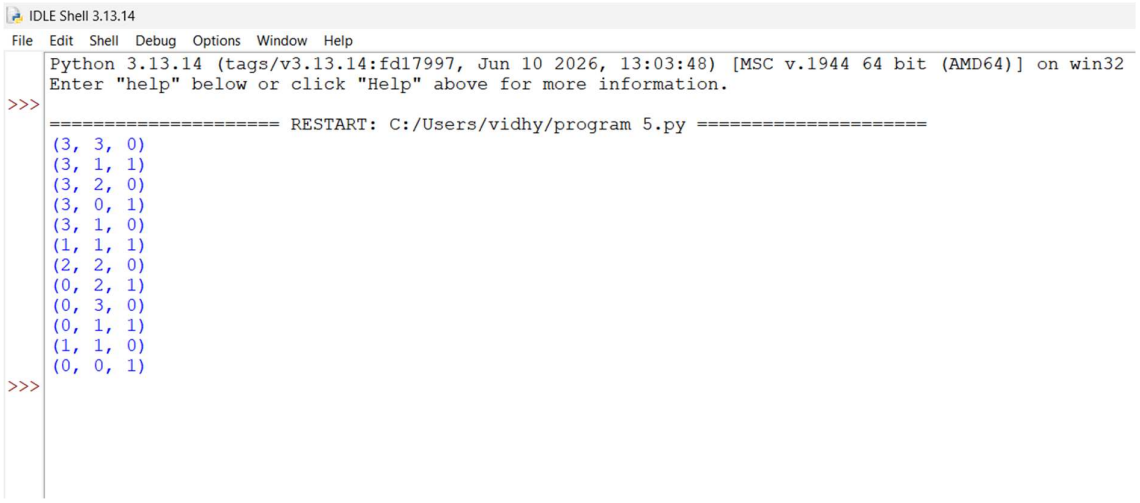
while queue:
    state, path = queue.popleft()
    m, c, boat = state

    if state == goal:
        for s in path:
            print(s)
        break

    for dm, dc in moves:
        if boat == 0:
            new = (m - dm, c - dc, 1)
        else:
            new = (m + dm, c + dc, 0)

        if valid(new[0], new[1]) and new not in visited:
            visited.add(new)
            queue.append((new, path + [new]))
```

Output:



Result: Thus, the Missionaries and Cannibals Problem was solved using Breadth-First Search (BFS).

Program 6. Vacuum Cleaner Problem

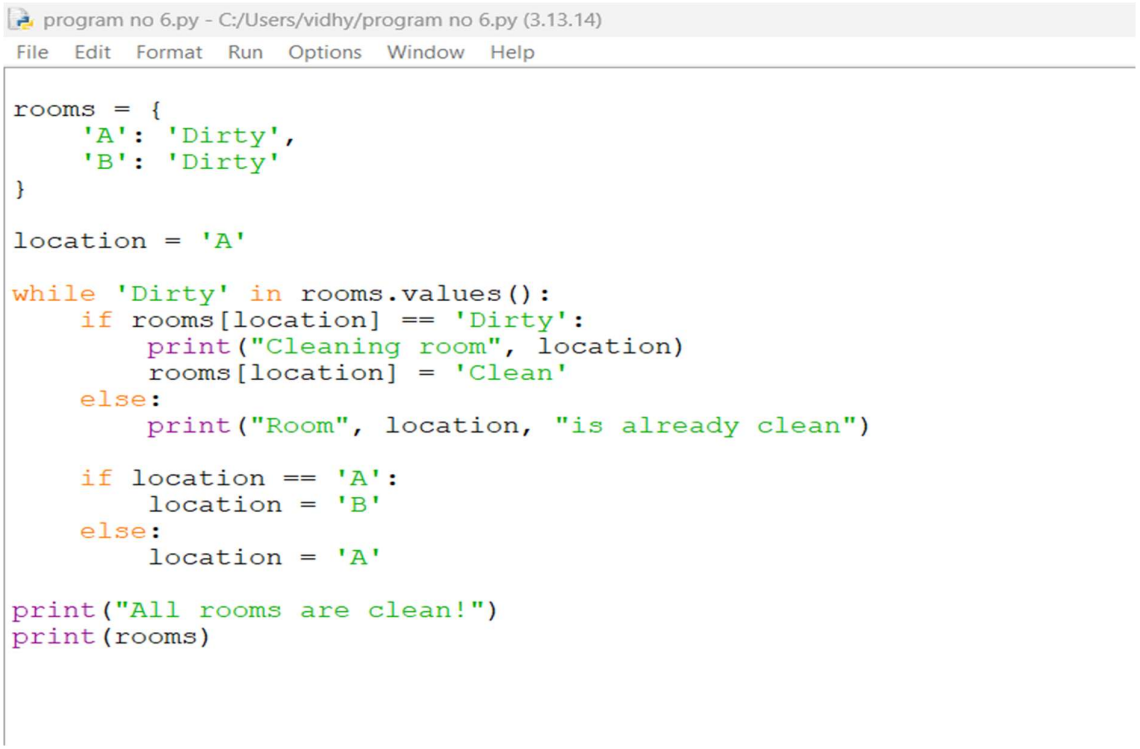
Aim:

To implement a simple vacuum cleaner agent that cleans dirty rooms.

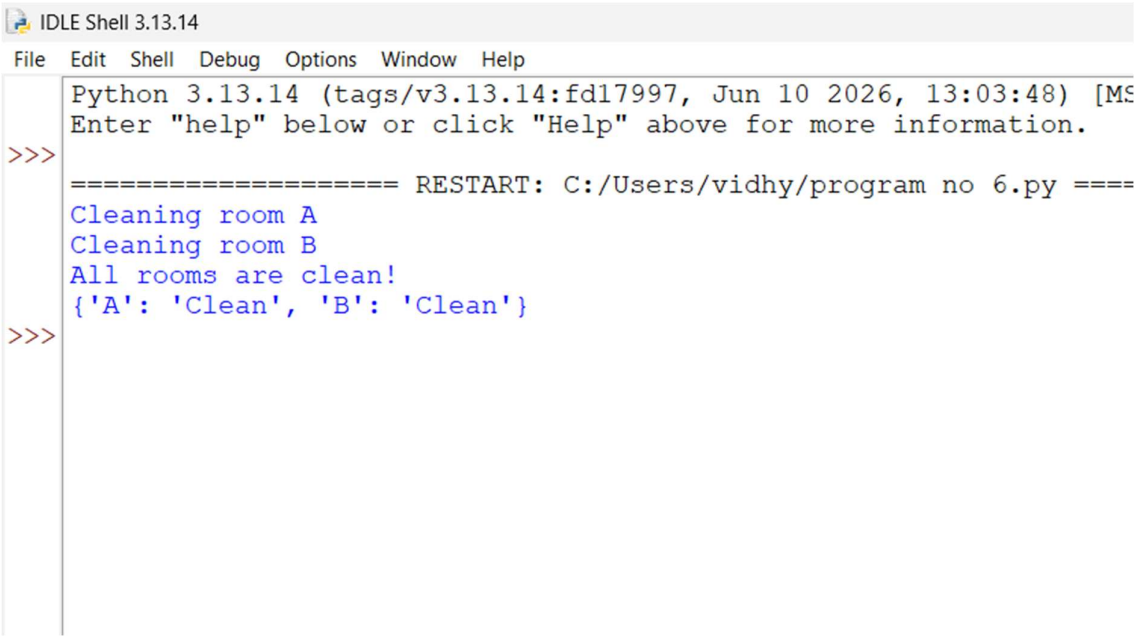
Algorithm:

1. Start the program.
2. Initialize the status of two rooms.
3. Check whether each room is dirty.
4. If dirty, clean the room.
5. Display the room status.
6. Stop the program.

Coding:


program no 6.py - C:/Users/vidhy/program no 6.py (3.13.14)
File Edit Format Run Options Window Help
rooms = {
'A': 'Dirty',
'B': 'Dirty'
}
location = 'A'
while 'Dirty' in rooms.values():
if rooms[location] == 'Dirty':
print("Cleaning room", location)
rooms[location] = 'Clean'
else:
print("Room", location, "is already clean")
if location == 'A':
location = 'B'
else:
location = 'A'
print("All rooms are clean!")
print(rooms)

Output:



Result:

Thus, the Vacuum Cleaner Problem was implemented successfully using Python.

Program7. Breadth-First Search (BFS)

Aim:

To implement Breadth-First Search to traverse a graph level by level.

Algorithm:

1. Start the program.
2. Create a graph using an adjacency list.
3. Insert the starting node into a queue.
4. Remove a node from the queue and display it.
5. Add its unvisited adjacent nodes to the queue.
6. Repeat until the queue is empty.
7. Stop the program.

Coding:

program no 7.py - C:/Users/vidhy/program no 7.py (3.13.14)

File Edit Format Run Options Window Help

```
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

def bfs(start):
    visited = set()
    queue = deque([start])

    while queue:
        node = queue.popleft()

        if node not in visited:
            print(node, end=" ")
            visited.add(node)

            for neighbour in graph[node]:
                if neighbour not in visited:
                    queue.append(neighbour)

bfs('A')
```

Output:

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48)
 (AMD64)] on win32
Enter "help" below or click "Help" above for more information
>>>
========== RESTART: C:/Users/vidhy/program no 7.py =
A B C D E F
>>>
```

Result:

Thus, BFS graph traversal was implemented successfully using Python.

Program 8. Depth-First Search (DFS)

Aim:

To implement Depth-First Search to traverse a graph by exploring one path as deeply as possible before backtracking**.**

Algorithm:

1. Start the program.
2. Create a graph using an adjacency list.
3. Mark the starting node as visited.
4. Display the current node.
5. Recursively visit each unvisited adjacent node.
6. Stop when all reachable nodes are visited.

Coding:

program no 8.py - C:/Users/vidhy/program no 8.py (3.13.14)

File Edit Format Run Options Window Help

```
graph = {
     'A': ['B', 'C'],
     'B': ['D', 'E'],
     'C': ['F'],
     'D': [],
     'E': [],
     'F': []
}

visited = set()

def dfs(node):
     if node not in visited:
          print(node, end=" ")
          visited.add(node)

          for neighbour in graph[node]:
               dfs(neighbour)

dfs('A')
```

Output:

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48)
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/vidhy/program no 8.py ==========
A B D E C F
>>>
```

Result:

Thus, DFS graph traversal was implemented successfully using Python.

Program 9. Travelling Salesman Problem (TSP)

Aim:

To find the minimum-cost tour that visits every city exactly once and returns to the starting city.

Algorithm:

1. Start the program.
2. Define the distance matrix for all cities.
3. Generate all possible orders of visiting the remaining cities.
4. Calculate the total distance of each tour.
5. Select the tour with the minimum distance.
6. Display the shortest tour and its cost.
7. Stop the program.

Coding:

program no 9.py - C:/Users/vidhy/program no 9.py (3.13.14)

File Edit Format Run Options Window Help

```
from itertools import permutations

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

n = len(graph)
cities = range(1, n)

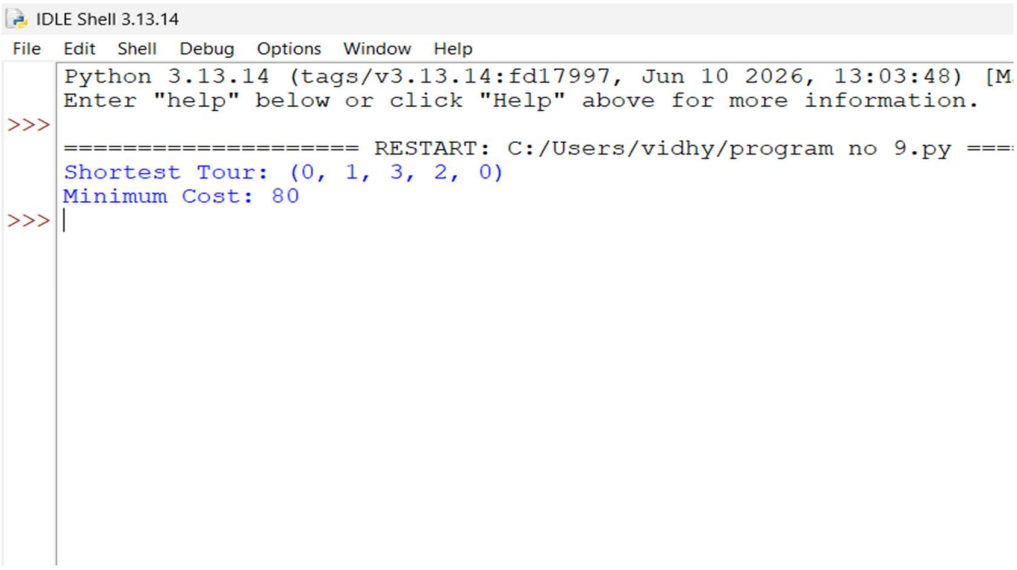
min_cost = float('inf')
best_path = []

for path in permutations(cities):
    tour = (0,) + path + (0,)
    cost = sum(
        graph[tour[i]][tour[i + 1]]
        for i in range(n)
    )

    if cost < min_cost:
        min_cost = cost
        best_path = tour

print("Shortest Tour:", best_path)
print("Minimum Cost:", min_cost)
```

Output:



Result:

Thus, the Travelling Salesman Problem was solved using permutations to find the minimum-cost tour.

Program 10. A* Search Algorithm

Aim:

To implement the A* search algorithm to find a path from a start node to a goal node using path cost and heuristic values.

Algorithm:

1. Start the program.
2. Define the graph with edge costs.
3. Assign heuristic values to each node.
4. Initialize the start node with path cost zero.
5. Select the node with the lowest f(n)=g(n)+h(n)
6. Update the path costs of its neighbours.
7. Continue until the goal node is reached.
8. Display the shortest path and its cost.

Coding:

program no 10.py - C:/Users/vidhy/program no 10.py (3.13.14)

File Edit Format Run Options Window Help

```
import heapq

graph = {
    'A': {'B': 1, 'C': 4},
    'B': {'C': 2, 'D': 5},
    'C': {'D': 1},
    'D': {}
}

h = {'A': 4, 'B': 3, 'C': 1, 'D': 0}

def astar(start, goal):
    pq = [(h[start], 0, start, [start])]
    visited = {}

    while pq:
        f, cost, node, path = heapq.heappop(pq)

        if node == goal:
            print("Path:", path)
            print("Cost:", cost)
            return

        if node in visited and visited[node] <= cost:
            continue

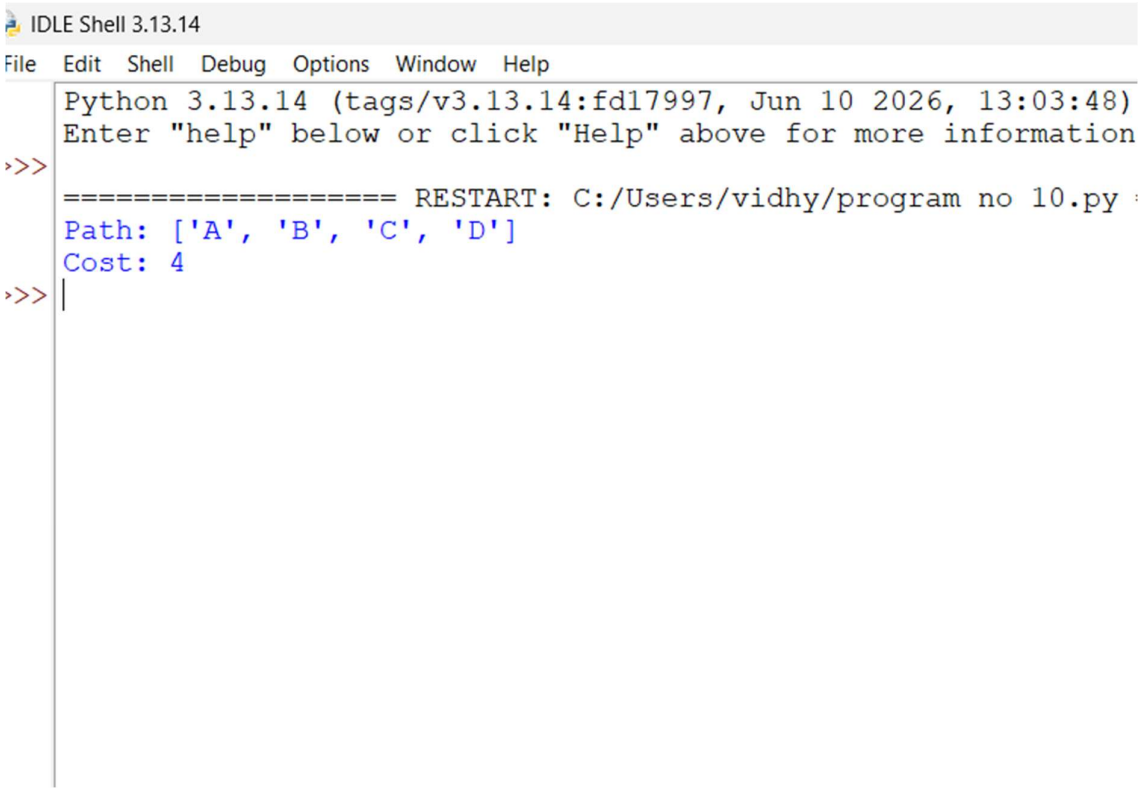
        visited[node] = cost

        for neighbour, distance in graph[node].items():
            new_cost = cost + distance
            new_f = new_cost + h[neighbour]

            heapq.heappush(
                pq,
                (new_f, new_cost, neighbour,
                 path + [neighbour])
            )

astar('A', 'D')
```

Output:



Result:

Thus, the A* search algorithm was implemented successfully to find the least-cost path from the start node to the goal node.